

Dijkstra's Algorithm

Dijkstra's Shortest-Path Algorithm

Goal

- Find a minimum weight path from a source node s to every other node in a graph $G = (V, E)$ with non-negative edge weights

Key idea

- Maintains a set S of vertices whose final shortest-path weights from the source s have already been determined
- Repeatedly selects the vertex $u \in V - S$ with the minimum shortest-path estimate, adds u to S , and relaxes all edges leaving u

Greedy algorithm

- Greedy choice: At each step, choose the vertex nearest to the source
 - Executes essentially the same as Prim's algorithm

```
Initialize-Single-Source( $G, s$ )
1. for each vertex  $v \in G.V$ 
2.      $v.d = \infty$  // distance to  $v$ 
3.      $v.\pi = \text{NIL}$  // predecessor of  $v$ 
4.  $s.d = 0$ 
```

```
Dijkstra( $G, w, s$ )
//  $S$ : explored vertices
// key: shortest path estimate  $*.d$ 
1. Initialize-Single-Source( $G, s$ )
2.  $S = \emptyset$ 
3.  $Q = G.V$ 
4. while  $Q \neq \emptyset$ 
5.      $u = \text{Extract-Min}(Q)$ 
6.      $S = S \cup \{u\}$ 
7.     for each vertex  $v \in G.\text{Adj}[u]$ 
8.         Relax( $u, v, w$ )
```

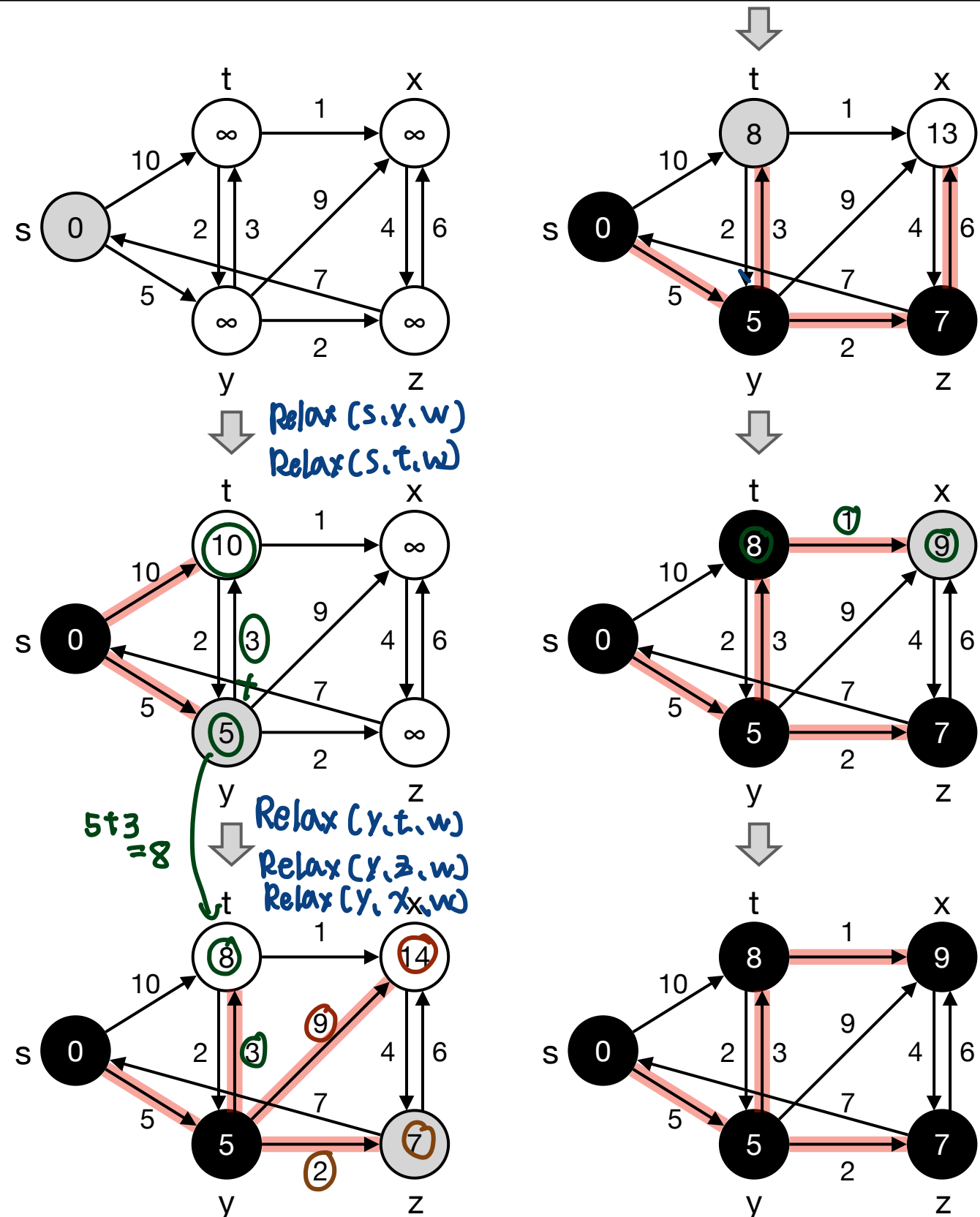
```
MST-Prim( $G, w, r$ )
1. for each vertex  $u \in G.V$ 
2.      $u.\text{key} = \infty$ 
3.      $u.\pi = \text{NIL}$ 
4.  $r.\text{key} = 0$ 
5.  $Q = G.V$ 
6. while  $Q \neq \emptyset$ 
7.      $u = \text{ExtractMin}(Q)$ 
8.     for each vertex  $v \in G.\text{Adj}[u]$ 
9.         if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10.             $v.\pi = u$ 
11.             $v.\text{key} = w(u, v)$ 
```

Example

```

Dijkstra(G, w, s)
// S: explored vertices
// key: shortest path estimate *.d
1. Initialize-Single-Source(G, s)
2. S =  $\Phi$ 
3. Q = G.V
4. while Q  $\neq \Phi$ 
5.     u = Extract-Min(Q)
6.     S = S  $\cup$  {u}
7.     for each vertex v  $\in$  G.Adj[u]
8.         Relax(u, v, w)
    
```

- Vertices in S (explored)
- Vertices u (processed in the while loop)
- Vertices in V - S (in Q)
- Indicate predecessor values



Practice

```

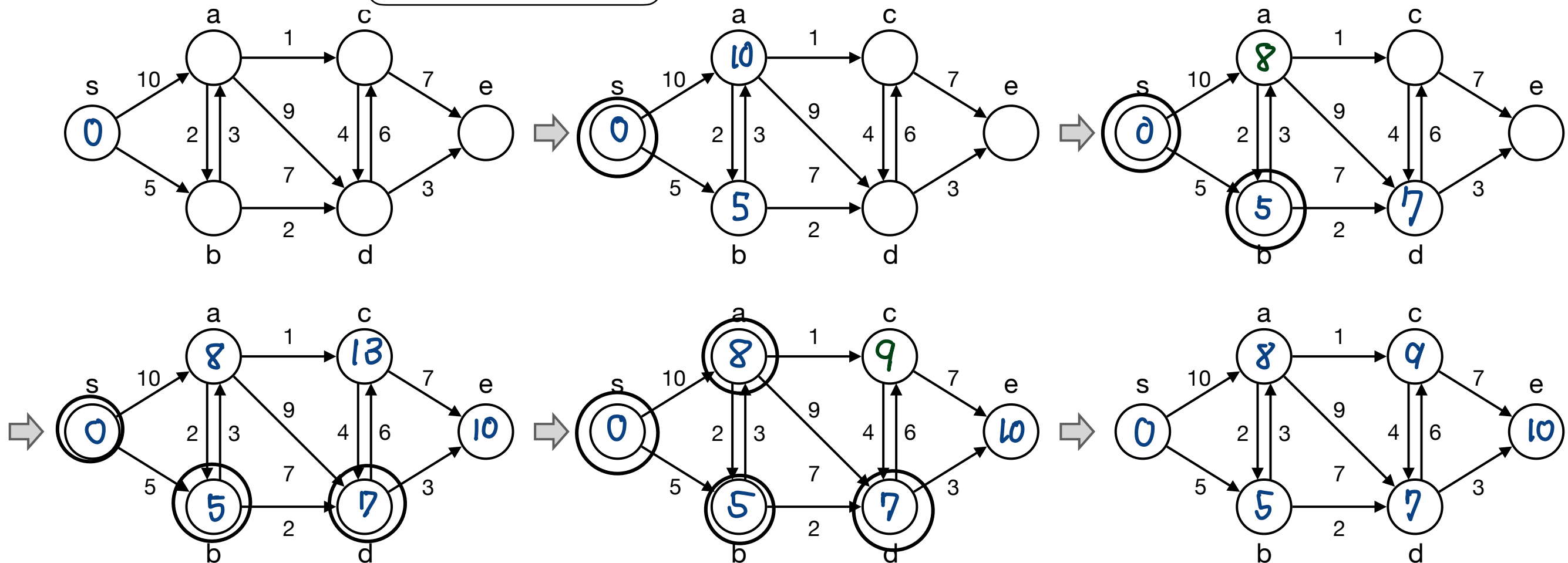
Initialize-Single-Source( $G, s$ )
1. for each vertex  $v \in G.V$ 
2.    $v.d = \infty$  // distance to  $v$ 
3.    $v.\pi = \text{NIL}$  // predecessor of  $v$ 
4.  $s.d = 0$ 
    
```

```

Relax( $u, v, w$ )
1. if  $v.d > u.d + w(u, v)$ 
2.    $v.d = u.d + w(u, v)$ 
3.    $v.\pi = u$ 
    
```

```

Dijkstra( $G, w, s$ )
//  $S$ : explored vertices
// key: shortest path estimate  $*.d$ 
1. Initialize-Single-Source( $G, s$ )
2.  $S = \emptyset$ 
3.  $Q = G.V$ 
4. while  $Q \neq \emptyset$ 
5.    $u = \text{Extract-Min}(Q)$ 
6.    $S = S \cup \{u\}$ 
7.   for each vertex  $v \in G.\text{Adj}[u]$ 
8.     Relax( $u, v, w$ )
    
```



Time Complexity

```
Dijkstra(G, w, s)
// S: explored vertices
// key: shortest path estimate *.d
1. Initialize-Single-Source(G, s)
2. S =  $\Phi$ 
3. Q = G.V
4. while Q  $\neq \Phi$ 
5.     u = Extract-Min(Q)
6.     S = S  $\cup$  {u}
7.     for each vertex v  $\in$  G.Adj[u]
8.         Relax(u, v, w)
```

Q is implemented as a **linear array**: $O(V^2)$

- Line 5: $O(V)$ for **Extract-Min**, so $O(V^2)$ with the **while** loop
- Line 7-8: $O(E)$ operations, each takes $O(1)$ time

Q is implemented as a **binary heap**: $O(E \lg V)$

- Line 5: $O(\lg V)$ for **Extract-Min**, so $O(V \lg V)$ with the **while** loop
- Line 7-8: $O(E)$ operations, each takes $O(\lg V)$ time for **Decrease-Key** (maintaining the heap property after changing the value of a node)

Q is implemented as a **Fibonacci heap**

- Amortized cost: $O(E + V \lg V)$

Correctness

Loop invariant

- Consider the set S during the algorithm's execution
 - For each node $v \in S$, $v.d = \delta(s, v)$

Proof by induction on $|S|$

- Basis step: Trivial for $|S| = 1$
- Inductive step
 - Hypothesis: True for iteration $k \geq 1$ (S contains k nodes)
 - Grow S by adding v ; let (u, v) be the final edge on our s - v path P_v
 - By induction hypothesis, P_u is the shortest s - u path
 - Consider any other s - v path P ; P must leave S somewhere; let y be the first node on P that is not in S , and $x \in S$ be the node just before y
 $P_v \leq P_y + P$
 - P cannot be shorter than P_v because P is already at least as long as P_v by the time it has left the set S since P_v is the shortest path leaving S
 - At iteration $k+1$, $w(P_v) = v.d = u.d + w(u, v) \leq x.d + w(x, y) = P_y \leq w(P) = w(P_y + P_{yv})$

```
Dijkstra(G, w, s)
// S: explored vertices
// key: shortest path estimate *.d
1. Initialize-Single-Source(G, s)
2. S =  $\emptyset$ 
3. Q = G.V
4. while Q  $\neq \emptyset$ 
5.     u = Extract-Min(Q)
6.     S = S  $\cup$  {u}
7.     for each vertex v  $\in$  G.Adj[u]
8.         Relax(u, v, w)
```

